



An adaptive DNN inference acceleration framework with end-edge-cloud collaborative computing

Guozhi Liu^a, Fei Dai^{a,*}, Xiaolong Xu^b, Xiaodong Fu^c, Wanchun Dou^d, Neeraj Kumar^{e,f,g,h}, Muhammad Bilal^{i,*}

^a School of Big Data and Intelligence Engineering, Southwest Forestry University, Kunming 650224, China

^b School of Computer and Software, Nanjing University of Information Science and Technology, Nanjing 210044, China

^c Faculty of Information Engineering and Automation, Kunming University of Science and Technology, Kunming 650500, China

^d State Key Laboratory for Novel Software Technology, Department of Computer Science and Technology, Nanjing University, 210008, China

^e Department of Computer Science and Engineering, Thapar Institute of Engineering & Technology, Patiala (Pb.), India

^f School of Computer Science, University of Petroleum and Energy Studies, Dehradun, Uttarakhand, India

^g Department of Electrical and Computer Engineering, Lebanese American University, Beirut, Lebanon

^h Faculty of Computing and IT, King Abdul Aziz University, Jeddah, Saudi Arabia

ⁱ Computer and Electronics Systems Engineering, Hankuk University of Foreign Studies, Yongin-si, Gyeonggi-do, 17035, Korea (the Republic of)

ARTICLE INFO

Article history:

Received 19 June 2022

Received in revised form 27 October 2022

Accepted 31 October 2022

Available online 4 November 2022

Keywords:

Deep neural networks

DNN inference acceleration

End-edge-cloud collaboration

DNN computation partitioning

Latency prediction model

ABSTRACT

Deep Neural Networks (DNNs) based on intelligent applications have been intensively deployed on mobile devices. Unfortunately, resource-constrained mobile devices cannot meet stringent latency requirements due to a large amount of computation required by these intelligent applications. Both existing cloud-assisted DNN inference approaches and edge-assisted DNN inference approaches can reduce end-to-end inference latency through offloading DNN computations to the cloud server or edge servers, but they suffer from unpredictable communication latency caused by long wide-area massive data transmission or performance degeneration caused by the limited computation resources. In this paper, we propose an adaptive DNN inference acceleration framework, which accelerates DNN inference by fully utilizing the end-edge-cloud collaborative computing. First, a latency prediction model is built to estimate the layer-wise execution latency of a DNN on different heterogeneous computing platforms, which use neural networks to learn non-linear features related to inference latency. Second, a computation partitioning algorithm is designed to identify two optimal partitioning points, which adaptively divide DNN computations into end devices, edge servers, and the cloud server for minimizing DNN inference latency. Finally, we conduct extensive experiments on three widely-adopted DNNs, and the experimental results show that our latency prediction models can improve the prediction accuracy by about 72.31% on average compared with four baseline approaches, and our computation partitioning approach can reduce the end-to-end latency by about 20.81% on average against six baseline approaches under three wireless networks.

© 2022 Elsevier B.V. All rights reserved.

1. Introduction

1.1. Background & motivation

Deep Neural Networks (DNNs) have been intensively deployed in various intelligent applications, such as video recognition [1], recommender systems [2], and information retrieval [3]. DNN-based applications have become increasingly popular as the core

on-board intelligent applications as end devices (i.e., mobile devices and wearable devices) become more powerful. Unfortunately, these intelligent applications are inherently computation-intensive and latency-critical inference tasks [4,5]. As a result, it is challenging to execute them on resource-limited end devices with low latency [6–8].

Generally, considering that DNN-based applications need numerous computations, cloud computing and edge computing are two proper computing paradigms to meet the stringent latency requirements. By partitioning DNN computations between end devices and the cloud server or between end devices and edge servers for collaborative execution, cloud-assisted DNN inference [9–11] or edge-assisted DNN inference [6,12,13] can reduce the inference latency of intelligent applications. In these

* Corresponding authors.

E-mail addresses: GuozhiLiu@swfu.edu.cn (G. Liu), daifei@swfu.edu.cn (F. Dai), xlxu@ieee.org (X. Xu), xiaodong_fu@hotmail.com (X. Fu), douwc@nju.edu.cn (W. Dou), nehra04@gmail.com (N. Kumar), bilal@ieee.org (M. Bilal).

collaborative computing approaches, DNN computations can be performed on end devices or offloaded to cloud servers or edge servers for computing. Unfortunately, cloud-assisted DNN inference approaches may result in unpredictable communication latency caused by the long wide-area massive data transmission between devices and the cloud server [7,14]. On the other hand, edge-assisted DNN inference approaches cannot continuously meet stringent real-time requirements [6,9,15] due to excessive computation of large-scale DNN and edge servers with limited computing capability.

To alleviate the above problems, it is naturally required to promote the cooperation of end devices, edge servers, and the cloud server, called end-edge-cloud collaborative computing. In this computing paradigm, DNN computations are partitioned between end devices, edge servers, and the cloud server for collaborative execution. This collaborative computing paradigm combines cloud computing with high computing capacity and edge computing with low transmission latency [7]. Therefore, the time of DNN inference is further reduced.

1.2. Challenges

However, accelerating DNN inference with end-edge-cloud collaborative computing still faces the following two challenges.

The first challenge is computation partitioning, which is the basis of collaborative computing [14,16]. Most of the existing computation partitioning approaches are cloud-assisted DNN inference [17] and edge-assisted DNN inference [18], which mainly use one partitioning point to divide DNN computations into end devices and the cloud server or into end devices and edge servers. Unfortunately, both these two approaches may lead to poor performance (i.e., high end-to-end latency) [6]. A comparison of the one-point partitioning mechanism and two-point partitioning mechanism is illustrated in Fig. 1. In this example, we employ a smartphone (i.e., Huawei nova 7 Pro), a laptop, and a server to emulate the mobile device, edge server, and cloud server respectively, and perform image recognition task over the ImageNet dataset with the classical AlexNet model, where the input image size is 224×224 . Specifically, Fig. 1a shows the layer-wise execution latency (i.e., the orange bar) and the intermediate output data (i.e., the blue bar) of each layer based on one-point partitioning mechanism. The best partition point is pool5, which means executing the AlexNet on the end device up to that layer and executing the remaining layers on the cloud server. Fig. 1b shows two-point partitioning mechanism has relatively 12.11% lower end-to-end inference latency than the one-point partitioning mechanism. Obviously, the two-point partitioning mechanism is more efficient than the one-point partitioning mechanism in the end-edge-cloud computing platforms.

Another challenge is per-layer latency prediction, which refers to accurately predicting the execution latency of each layer in a DNN before computation partitioning. Most of the existing prediction approaches [19,20] rely on linear models to estimate each layer's execution delay. Nevertheless, the prediction accuracy is highly related to the computing platforms and the DNN topology. Compared with the end-edge or end-cloud computing platforms, end-edge-cloud computing platforms make the prediction process more difficult due to complex heterogeneity. Therefore, a more efficient prediction model will be needed for better prediction performance.

1.3. Contributions of the work

Inspired by these observations, in this paper, we propose an adaptive DNN inference acceleration framework to reduce DNN inference latency with end-edge-cloud computing, which

adaptively partitions DNN computations into end devices, edge servers, and the cloud server for executions based on layer prediction results. Specifically, the main contributions of this article are summarized as follows:

- Build latency prediction models to estimate the layer-wise execution latency of a DNN on different heterogeneous computing platforms, which uses neural networks to learn non-linear features related to inference latency for better prediction performance.
- Design a computation partitioning algorithm to identify two optimal partitioning points, which divides DNN computations into three blocks (i.e., data-intensive block, hybrid block, and computation-intensive block) for accelerating DNN inference across end devices, edge servers, and the cloud server.
- Conduct extensive experiments on three widely-adopted DNNs to evaluate the effectiveness of the DNN inference acceleration framework. The experimental results show that the proposed framework can greatly reduce DNN inference latency compared with the five baseline approaches.

The rest of this paper is organized as follows. Section 2 discusses the related work. Section 3 presents the details of the adaptive DNN inference acceleration framework. Section 4 evaluates the effectiveness of the DNN inference acceleration framework. Section 5 concludes this paper and outlines future work.

2. Related work

Collaborative DNN inference approaches have been extensively studied, which can be roughly classified into cloud-assisted DNN inference [20], edge-assisted DNN inference [6], and collaborative end-edge-cloud DNN inference [14].

Cloud-assisted DNN inference refers to a collaborative computing approach, which partitions DNN computations into end devices and the cloud server for accelerating DNN inference. Kang et al. [20] studied the computing and data properties of 8 neural network-based intelligent applications and proposed a system to reduce DNN inference delay. Baccarelli et al. [21] considered both the supervised training and energy-efficient of the Conditional DNNs (CDNNs) with early-exits. A greedy-based algorithm was proposed to place the early exits on the CDNNs. Eshratifar et al. [10] proposed an efficient engine for collaborative computation between end devices and remote cloud, where an algorithm based on the integer linear programming (ILP) was developed to minimize inference delay and mobile energy consumption. Duan et al. [22] proposed a binary-search-based algorithm to select the best partitioning point between end devices and the cloud to maximize the performance of DNN inference. Baccarelli et al. [23] considered the model early-exit to make a good balance between accuracy and delay. A Learning-in-the-Fog (LiFo) framework was designed to minimize energy and maximize DNN inference accuracy. Eshratifar et al. [24] design a optimal task offloading approach to reduce the inference latency and mobile energy consumption while satisfying latency needs, in which both inference time and mobile energy consumption were considered. Baccarelli et al. [25] suggested a task offloading scheme for joint optimization of the resource allocation and task offloading in 5G, where an ad-hoc-developed Genetic algorithm was proposed to reasonably allocate resources and reduce the energy consumption of end devices. However, these works are highly susceptible to data transmission latency between end devices and the cloud server. Bandwidth congestion may incur unstable data transmission and then result in high response latency.

Edge-assisted DNN inference refers to a collaborative computing approach, which divides DNN computations into end devices and edge servers for execution. Li et al. [6] considered

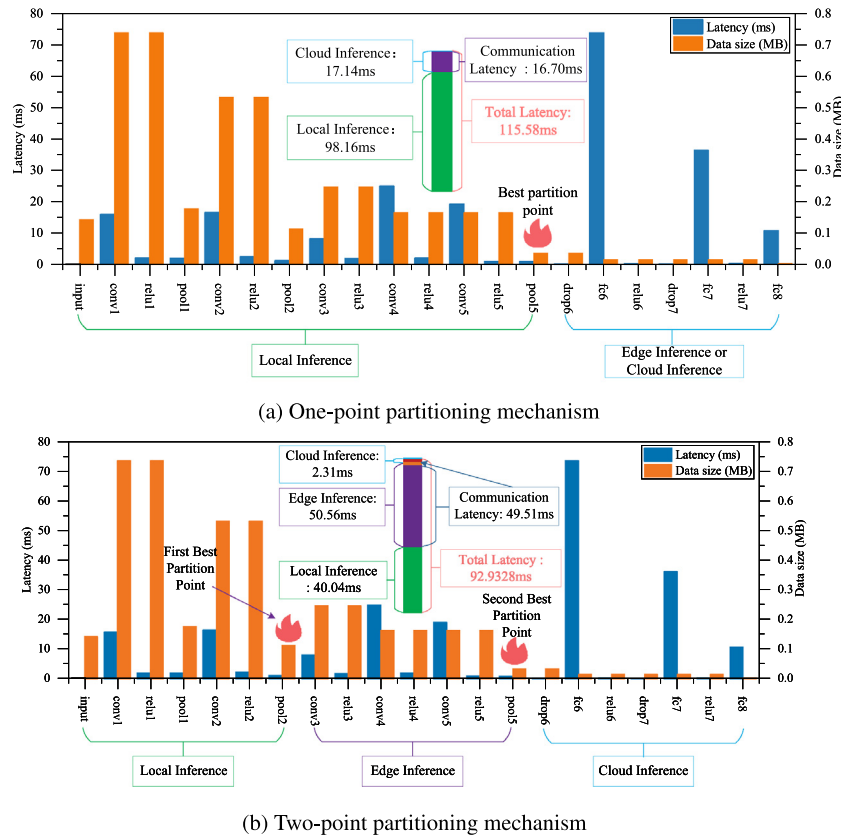


Fig. 1. Comparison of one-point partitioning mechanism and two-point partitioning mechanism.

both energy consumption and DNN inference latency in the end-edge environment and proposed an adaptive partition framework for reducing inference time and energy consumption under the end-edge environment. Zeng et al. [26] proposed a distributed DNN computing system to improve the performance of DNN inference by utilizing available computation and communication resources to dynamically offloading DNN computation into edge servers. Gao et al. [13] considered both computation delay and the interests of service provider in end-edge collaborative environment. A distributed algorithm with the aggregative game theory was proposed to reduce the computation delay and interests of service provider. Jeong et al. [27] suggested an incremental offloading of DNN scheme for accelerating DNN inference, where a graph-based algorithm was developed to select the best partitioning point between devices and the edge servers. However, these works use one-point partitioning mechanism to divide DNN computations into end devices and edge servers. Since these works did not make use of the rich resources of the cloud, the performance of these works is not excellent.

Collaborative end-edge-cloud DNN inference refers to a collaborative computing approach, which accelerates DNN inference by partitioning DNN computations into end devices, edge servers, and the cloud server. Ren et al. [14] design an effective partitioning scheme based on Deep Deterministic Policy Gradient, which can reduce both inference delay and energy consumption of end devices. Zhang et al. [28] investigate the problem of DNN inference in the end-edge-cloud computing environment with multi-dimensional resource constraints. A Deep Q-Network (DQN)-based resource scheduling algorithm was proposed to make real-time optimal resource allocation decisions. Lin et al. [29] proposed a genetic algorithm (GA) based partitioning optimization algorithm to reduce system cost under the end-edge-cloud environment. Where divides DNN inference

tasks between end, edge server, and the cloud for collaborative inference. Ren et al. [30] investigated the DNN dividing for end devices, edge servers, and the cloud cooperation and developed a DNN computation scheduling scheme to improve the intelligent applications in the 5G network. However, these works mainly use linear models to predicate layer-wise inference latency. Since the linear models cannot capture the complex non-linear features related to inference, the performance of these works is not excellent. In addition, these works are fine-grained DNN partitioning while our approach is coarse-grained DNN partitioning. The computational complexity of our approach is much less than that of these works.

Unlike these above works, we investigate the problem of accelerating DNN inference in the end-edge-cloud collaborative computing environment. Our work is different from these works in two aspects: (1) We use neural networks to build latency prediction models for better prediction performance on different heterogeneous computing platforms. (2) We utilize two-point partitioning mechanism to divide the DNN computations into end-edge-cloud computing platforms for better inference acceleration, which fully considers the collaborative computing of the end-edge-cloud computing platforms.

3. An adaptive DNN inference acceleration framework

In this section, we propose an adaptive DNN inference acceleration framework in the end-edge-cloud computing environment. First, we present the design of the framework. Second, we propose latency prediction models in the prediction stage. Third, we model inference latency and then propose a computation partitioning algorithm in the partitioning stage. Finally, we introduce the collaborative inference stage.

In Table 1, we give the main notations and corresponding descriptions in this paper.

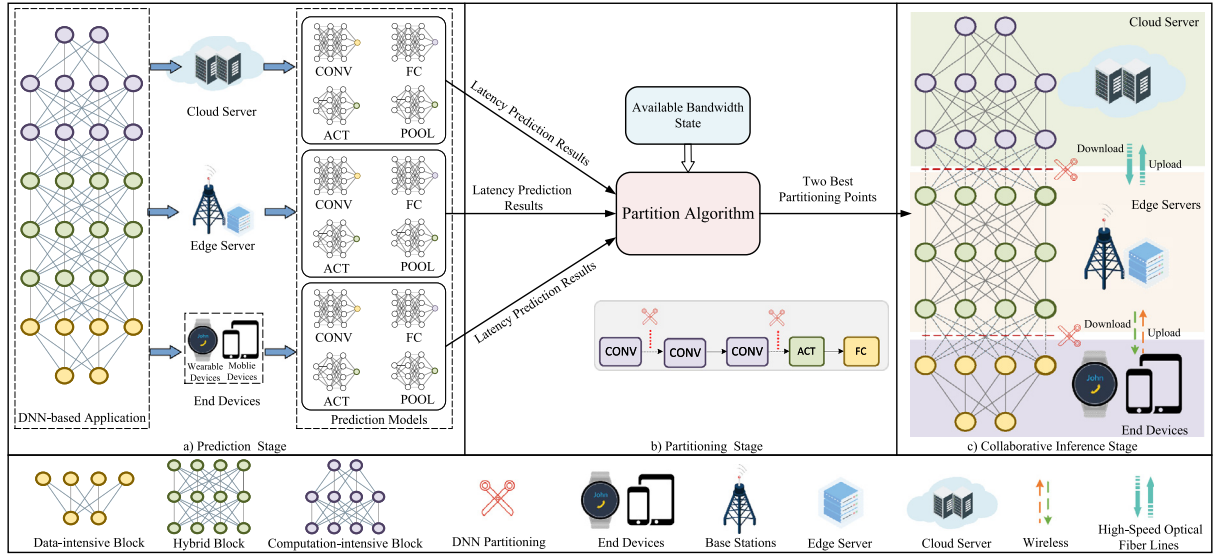


Fig. 2. Overview of framework.

Table 1
Main notations.

Notations	Descriptions
G_i^{input}	The input of the i th DNN block
G_i^{output}	The output of the i th DNN block
S	The collaboration strategy
$B_{end-edge}^U$	The uplink bandwidths (Mbps) between the end device and edge server
$B_{edge-cloud}^U$	The uplink bandwidths (Mbps) between the edge and cloud server
$t_{inference}^{end}$	The execution time on end devices
$t_{inference}^{edge}$	The execution time on the edge server
$t_{inference}^{cloud}$	The execution time on the cloud server
t_{total}	The total cost
B	The maximum bandwidth capability of the network
C	The maximum computing capabilities of the end-edge-cloud computing platform
β^E	The allocated communication resources
γ^E	The allocated computation resources
Q_{chain}	The chain DNN model
Q_{DAG}	The DAG (directed acyclic graph) DNN model

3.1. Framework design

The adaptive DNN inference acceleration framework is shown in Fig. 2, which consists of three stages, namely the prediction stage, partitioning stage, and collaborative inference stage.

At the prediction stage, this framework trains latency prediction models to estimate layer-wise inference latency of DNNs on three heterogeneous computing platforms (i.e., end devices, edge servers, and the cloud server). The latency prediction results are the basis for DNN computation partitioning.

At the partitioning stage, this framework uses a computation partitioning algorithm to divide DNN computations into three blocks, namely, data-intensive block, hybrid block, and computation-intensive block. This algorithm searches for the two best partitioning points by jointly considering layer-wise prediction results and the dynamic network condition.

At the collaborative inference stage, this framework offloads the hybrid block and the computation-intensive block to the edge server and the cloud server for inference acceleration respectively while the data-intensive block is executed on the end device for reducing data transmission latency.

3.2. Prediction stage

This stage has three main tasks: profiling execution latency, training latency prediction models, and utilizing latency prediction models.

(1) Profiling execution latency. This task is to predicate the execution latency of each type of layer on different computing platforms. Since the execution latency is platform-dependent and type-dependent, the profiling process should consider different computing platforms and layer types. Specifically, four types of DNN layers are considered in the framework, namely convolution (CONV), fully-connected (FC), activation (ACT), and pooling layer (POOL). Meanwhile, three different computing platforms (i.e., end devices, edge servers, and the cloud server) are taken into consideration in the task.

(2) Training latency prediction models. The task is to train different neural networks on various computing platforms and layer types for layer-wise inference prediction. Since a DNN has limited types of layers, the training cost is moderate. In addition, we choose neural networks to build latency prediction models rather than linear models because the neural networks can learn well non-linear features related to execution latency.

- For the prediction models of convolution and full-connected layers, we consider the input data size d_{input} , the output data size d_{output} , the size of input feature maps f_{input} , the size of output feature maps f_{output} , the memory speed of computing platform $m \in \{m_{end}, m_{edge}, m_{cloud}\}$, and the computing capability $c \in \{c_{end}, c_{edge}, c_{cloud}\}$ of computing platform in the training process.
- For the prediction models of activation and pooling layers, we consider the input data size d_{input} , the input feature maps f_{input} , the size of output feature maps f_{output} , the memory speed of computing platform $m \in \{m_{end}, m_{edge}, m_{cloud}\}$, and the computing capability $c \in \{c_{end}, c_{edge}, c_{cloud}\}$ of computing platform in the training process. Since the output data size and the output feature maps of these two layers are equal to the input data size and the input feature maps respectively, the training data with two features are needed to construct for training these prediction models.

(3) Utilizing latency prediction models. This task is to use trained latency prediction models to estimate the execution latency of each type of layers on different computing platforms.

Four types of DNN layers and three computing platforms are considered in the task.

3.3. Partitioning stage

This stage has two main tasks: modeling inference latency, and searching partitioning points.

(1) Modeling inference latency. This task is to define the inference latency of a DNN block rather than a DNN layer on three different computing platforms. In this paper, each DNN block is composed of a sequence of layers.

To be precise, we let $L = \{L_1, L_2, \dots, L_N\}$ denote the DNN topology composing of a sequence of layers, let $G = \{G_1, G_2, G_3\}$ denote the three DNN blocks, and let G_i^{input} and G_i^{output} denote the input and output of the i th DNN block, respectively. Moreover, we let $S = \{S_1, S_2, S_3\}$ denote the collaboration strategy, where $S_i \in \{1, 2, 3\}$. S_i is the indicator that describes whether the i th DNN block is executed on the end device, the edge server, or the cloud layer, respectively, that is, $S_i = 1$ means the block is performed on the end devices; $S_i = 2$ indicates that the block is offloaded to the edge server for execution; and $S_i = 3$ denotes that the block is offloaded to the cloud server for execution.

(2) Searching partitioning points. Based on the inference latency of a DNN block, we search for the two best partitioning points to minimize the end-to-end inference latency of a DNN on an end-edge-cloud computing platform. Note that the layer-by-layer inference mechanism [14,31] provides execution semantics for DNN partitioning, that is, the current DNN block can only be performed after the computing platform receives the output of the previous DNN block; and only after executing the current DNN block, the computing platform will send the output to the next DNN block.

Uses Algorithms 1 and 2 to output the two optimal partitioning points based on the layer-wise prediction results and the current network condition, which adaptively divides DNN computations into end devices, edge servers, and the cloud server.

3.3.1. Modeling inference latency

Since the end-edge-cloud computing platform provides three infrastructures for executing a specific DNN block, the inference latency of the DNN block should include three types of latency: local inference latency, edge inference latency, and cloud inference latency.

We let $B_{end-edge}^U$ and $B_{end-edge}^D$ denote the uplink and downlink bandwidths (Mbps) between the end devices and the edge server, respectively, and let $B_{edge-cloud}^U$ and $B_{edge-cloud}^D$ denote the uplink and downlink bandwidths (Mbps) between the edge server and the cloud server, respectively.

(1) Local inference latency. When the i th DNN block is assigned to be executed on the end devices (i.e., $S_i = 1$), the local inference latency refers to the local execution time of all layers in the block, which is defined as

$$t_{inference}^{end} = \sum_{i=1, S_i=1}^N t_i^{end} \quad (1)$$

where N denotes the number of layers in i th DNN block and t_i^{end} denotes the local execution time of the i th layer.

(2) Edge inference latency. When the i th DNN block is assigned to be offloaded to the edge server for execution, the edge inference latency can be divided into two parts: end data transmission latency and edge execution latency, which are defined as

$$t_{inference}^{edge} = \frac{G_i^{input}}{B_{end-edge}^U} + \sum_{i=1, S_i=2}^N t_i^{edge} \quad (2)$$

where $\frac{G_i^{input}}{B_{end-edge}^U}$ denotes the end data transmission latency between the end devices and the edge server and $\sum_{i=1, S_i=2}^N t_i^{edge}$ denotes the edge execution time of all layers in the i th DNN block.

Note that when the first DNN block is offloaded to the edge server for processing, G_i^{input} refers to the raw input, which needs to be transferred from the end devices to the edge server.

(3) Cloud inference latency. When the i th DNN block is assigned to be offloaded to the cloud server for execution, the cloud inference latency can be divided into three parts: end data transmission latency, edge data transmission latency, and cloud execution time, which are defined as

$$t_{inference}^{cloud} = \frac{G_i^{input}}{B_{end-edge}^U} + \frac{G_i^{input}}{B_{edge-cloud}^U} + \sum_{i=1, S_i=3}^N t_i^{cloud} \quad (3)$$

where $\frac{G_i^{input}}{B_{end-edge}^U}$ denotes the end data transmission latency between the end devices and the edge server, $\frac{G_i^{input}}{B_{edge-cloud}^U}$ denotes the edge data transmission latency between the edge server and the cloud server, and $\sum_{i=1, S_i=3}^N t_i^{cloud}$ denotes the cloud execution time of all layers in the i th DNN block.

3.3.2. Problem formulation

To better understand DNN computation partitioning on an end-edge-cloud computing platform, we formalize the problem of searching partitioning points as an optimization problem. This optimization problem aims to minimize the end-to-end inference latency of a DNN under the constraints of the network resources and the end-edge-cloud computing resources. Specifically, the optimization problem is defined as

$$t_{total} = \underset{\{x\}}{\operatorname{argmin}} (t_{inference}^{end} + t_{inference}^{edge} + t_{inference}^{cloud}) \quad (4)$$

$$\text{s.t.} \quad \beta \leq B \quad (5a)$$

$$\gamma \leq C \quad (5b)$$

where $x = \{\beta, \gamma\}$ denotes the allocated communication and computation resources, B denotes the maximum bandwidth capability of the network, and C denotes the maximum computing capabilities of the end-edge-cloud computing platform.

3.3.3. Searching partitioning points

In this task, the computation partitioning algorithm is proposed to identify the two best partitioning points using the enumerative approach. Considering the complex topologies of DNN models, we mainly focus on chain DNN models (e.g., AlexNet, Vgg16, and Vgg19) and DAG (directed acyclic graph) DNN models (e.g., ResNet, MobileNet, and GoogleNet) in this paper. Moreover, we let Q_{chain} and Q_{DAG} denote a chain DNN model and a DAG DNN model, respectively.

For a chain DNN model, the computation partitioning algorithm can be found in Algorithm 1. Algorithm 1 initializes four variables and then traverses all the layers in the chain DNN model to obtain the two best partitioning points (i.e., p_1 and p_2). During the two best partitioning points searching, the inference latency of three DNN blocks needs to be computed. Firstly, the inference latency of the first DNN block (i.e., $t_{inference,1}^{end}$) is computed on the end computing platform (lines 10–13), where the execution time is predicted by the trained prediction model. Secondly, the inference latency of the second DNN block is computed on the edge computing platform (lines 14–17), where the data transmission

latency between the end device and the edge server (i.e., $\frac{G_t^{\text{input}}}{B_{\text{end-edge}}^U}$) is considered. Finally, the inference latency of the third DNN block is computed on the cloud computing platform (lines 18–21), where the data transmission latency between the edge server and the cloud server (i.e., $\frac{G_j^{\text{input}}}{B_{\text{edge-cloud}}^U}$) is considered.

The time complexity of Algorithm 1 mainly consists of three parts. The first part is from lines 1 to 4. The computational complexity of this part is $O(1)$. The second part is from lines 5 to 29. Since it has three layers of loops, its computational complexity is $O(n^3)$. The remaining part is from lines 30 to 31 and its computational complexity is $O(1)$. Based on the above analysis, the total time complexity of Algorithm 1 is $O(n^3)$.

For a DAG DNN model, the computation partitioning algorithm is outlined in Algorithm 2. Compared with Algorithm 1, Algorithm 2 adds a loop (e.g., lines 12–13, lines 20–21, and lines 28–29). This is because each layer in the DAG DNN models is not atomic, which consists of multiple sublayers. Therefore, the execution latency of the k th layer is equal to the sum of the execution time of the R_k sublayers. Note that $t_{\text{execution},k,r}^{\text{end}}$ denotes the execution time of each sublayer.

Since Algorithm 2 has four layers of loops, its time complexity is $O(n^4)$.

3.4. Collaborative inference stage

The collaborative inference stage is shown in Fig. 2c, which consists of end devices, edge servers, and the cloud server.

- (1) **End Devices:** End devices refer to various intelligent mobile devices (e.g., smartphones, iPad, and ereaders) and intelligent wearable devices (e.g., smartwatches, wrist bands, and smart glasses) and communicate with edge servers through wireless network connections (e.g., 4G, 5G, and WiFi), which generate various computation-intensive and latency-critical DNN-based intelligent applications. In these applications, the data-intensive DNN layer with light computation can be executed on end devices in a way that fast responsive.
- (2) **Edge Servers:** Edge servers refer to equipment with limited computing capability in close proximity to end devices. The hybrid layer is between the data-intensive layer and the computation-intensive layer will be offloaded to the edge server for execution because the collaborative computing between devices and edge servers helps to make a good balance between computation inference and communication inference.
- (3) **Cloud Server:** Cloud server refers to equipment with sufficient computing resources and communicates with edge servers through high-speed optical fiber lines. The computation-intensive DNN layer with few data can be fast computing on the cloud server because the collaborative computing between edge servers and the cloud server can further improve DNN inference efficiency.

Specifically, the framework uses the obtained two best partitioning points to guide the collaborative inference across end devices, edge servers, and the cloud server, that is, after the DNN is divided into three blocks (i.e., data-intensive block, hybrid block, and computation-intensive block) according to the two partitioning points, this stage accelerates DNN inference through computation offloading. Specifically, the data-intensive block before the first partitioning point will be executed on end devices locally, the hybrid block before the second partitioning point will be offloaded to the edge server for partitioned execution, and the rest DNN block (i.e., computation-intensive block) will be offloaded to the cloud server.

Algorithm 1: Adaptive chain DNN Computation Partitioning Algorithm

Input: current monitored wireless network uplink bandwidth, $B_{\text{end-edge}}^U$;
current monitored wired network uplink bandwidth, $B_{\text{edge-cloud}}^U$;

Output: two partition points $P = \{p_1, p_2\}$;

```

1 Set the end-to-end latency as  $t_{\text{total}} = 0$ 
2 Set the first partition point as  $p_1 = 0$ 
3 Set the second partition point as  $p_2 = 0$ 
4 Set  $t_{\text{min}} = +\infty$ 
5 for  $i = 1 : N$  do
6   for  $j = i + 1 : N$  do
7     Set  $t_{\text{execution}}^{\text{end}} = 0$ 
8     Set  $t_{\text{execution}}^{\text{edge}} = 0$ 
9     Set  $t_{\text{execution}}^{\text{cloud}} = 0$ 
10    for  $k = 1 : i$  do
11       $t_{\text{execution}}^{\text{end}} = t_{\text{execution}}^{\text{end}} + t_{\text{execution},k}^{\text{end}}$ 
12    end
13     $t_{\text{inference},1}^{\text{end}} = t_{\text{execution}}^{\text{end}}$ 
14    for  $k = i + 1 : j$  do
15       $t_{\text{execution}}^{\text{edge}} = t_{\text{execution}}^{\text{edge}} + t_{\text{execution},k}^{\text{edge}}$ 
16    end
17     $t_{\text{inference},2}^{\text{edge}} = t_{\text{execution}}^{\text{edge}} + \frac{G_i^{\text{input}}}{B_{\text{end-edge}}^U}$ 
18    for  $k = j + 1 : N$  do
19       $t_{\text{execution}}^{\text{cloud}} = t_{\text{execution}}^{\text{cloud}} + t_{\text{execution},k}^{\text{cloud}}$ 
20    end
21     $t_{\text{inference},3}^{\text{cloud}} = t_{\text{execution}}^{\text{cloud}} + \frac{G_j^{\text{input}}}{B_{\text{edge-cloud}}^U}$ 
22     $t_{\text{total}} = t_{\text{inference},1}^{\text{end}} + t_{\text{inference},2}^{\text{edge}} + t_{\text{inference},3}^{\text{cloud}}$ 
23    if  $t_{\text{total}} < t_{\text{min}}$  then
24       $t_{\text{min}} = t_{\text{total}}$ 
25       $p_1 = i$ 
26       $p_2 = j$ 
27    end
28  end
29 end
30  $P = \{p_1, p_2\}$ 
31 output  $P$ 

```

4. Performance evaluation

In this section, we conduct extensive experiments to evaluate the effectiveness of the adaptive DNN inference acceleration framework.

4.1. Experimental setup

4.1.1. Specifications of end-edge-cloud computing platform

We use the Huawei nova 7 Pro as an end device. The detailed specifications of the end platform are shown in Table 2.

We use a PC as an edge server. The detailed specifications of the edge platform are shown in Table 3.

We utilize one server as cloud platform. The detailed specifications of the cloud platform are shown in Table 4.

4.1.2. Specifications of network

We consider three wireless networks (i.e., 4G, 5G, and Wi-Fi) and one wired network in the experiments. The wireless network performance between the end device and the edge server

Algorithm 2: Adaptive DAG DNN Computation Partitioning Algorithm

Input: current monitored wireless network uplink bandwidth, $B_{end-edge}^U$;
current monitored wired network uplink bandwidth, $B_{edge-cloud}^U$;

Output: two partition points $P = \{p_1, p_2\}$;

- 1 Set the end-to-end latency as $t_{total} = 0$
- 2 Set the first partition point as $p_1 = 0$
- 3 Set the second partition point as $p_2 = 0$
- 4 Set $t_{min} = +\infty$
- 5 **for** $i = 1 : N$ **do**
- 6 **for** $j = i + 1 : N$ **do**
- 7 Set $t_{execution}^{end} = 0$
- 8 Set $t_{edge}^{end} = 0$
- 9 Set $t_{cloud}^{end} = 0$
- 10 **for** $k = 1 : i$ **do**
- 11 Set $t_{execution,k}^{end} = 0$
- 12 **for** $r = 1 : R_k$ **do**
- 13 $t_{execution,k}^{end} = t_{execution,k}^{end} + t_{execution,k,r}^{end}$
- 14 **end**
- 15 $t_{execution}^{end} = t_{execution}^{end} + t_{execution,k}^{end}$
- 16 **end**
- 17 $t_{inference,1}^{end} = t_{execution}^{end}$
- 18 **for** $k = i + 1 : j$ **do**
- 19 Set $t_{edge,k}^{edge} = 0$
- 20 **for** $r = 1 : R_k$ **do**
- 21 $t_{edge,k}^{edge} = t_{edge,k}^{edge} + t_{edge,k,r}^{edge}$
- 22 **end**
- 23 $t_{edge}^{edge} = t_{edge}^{edge} + t_{edge,k}^{edge}$
- 24 **end**
- 25 $t_{inference,2}^{edge} = t_{edge}^{edge} + \frac{G_i^{input}}{B_{end-edge}^U}$
- 26 **for** $k = j + 1 : N$ **do**
- 27 Set $t_{cloud,k}^{cloud} = 0$
- 28 **for** $r = 1 : R_k$ **do**
- 29 $t_{cloud,k}^{cloud} = t_{cloud,k}^{cloud} + t_{cloud,k,r}^{cloud}$
- 30 **end**
- 31 $t_{cloud}^{cloud} = t_{cloud}^{cloud} + t_{cloud,k}^{cloud}$
- 32 **end**
- 33 $t_{inference,3}^{cloud} = t_{cloud}^{cloud} + \frac{G_j^{input}}{B_{edge-cloud}^U}$
- 34 $t_{total} = t_{inference,1}^{end} + t_{inference,2}^{edge} + t_{inference,3}^{cloud}$
- 35 **if** $t_{total} < t_{min}$ **then**
- 36 $t_{min} = t_{total}$
- 37 $p_1 = i$
- 38 $p_2 = j$
- 39 **end**
- 40 **end**
- 41 **end**
- 42 $P = \{p_1, p_2\}$
- 43 **output** P

is monitored by SpeedTest [32]. Similarly, the available wired network bandwidth between the edge server and the cloud server is also monitored by SpeedTest [32]. Since the network performance is occasionally unstable, we use the average value in the experiments. More detailed network specifications are shown in Table 5.

Table 2

End platform specifications.

Hardware	Specification
System	HarmonyOS 2.0.0
CPU	HUAWEI Kirin 985 (1 × Cortex-A76 Based 2.58 GHz + 3 × Cortex-A76 Based 2.40 GHz + 4 × Cortex-A55 1.84 GHz)
Memory	8 GB
HDD	128 GB

Table 3

Edge platform specifications.

Hardware	Specification
System	Windows 10
CPU	4 × Intel(R) Core (TM) CPU, 1.60 GHZ
Memory	Samsung 16 GB 2133 MHz
HDD	WDC PC SN730 512 GB
GPU	NVIDIA GeForce Max250 2 GB

Table 4

Cloud platform specifications.

Hardware	Specification
System	Windows 10
CPU	12 × Intel(R) Xeon(R) CPU E5-2678 2.50 GHz
Memory	Samsung 64 GB 2400 MHz
HDD	4 × WDC PC SN730 512 GB
GPU	3 × NVIDIA GeForce GTX 1080Ti

4.1.3. Benchmark of DNN architecture

In the experiments, we choose three representative neural network architectures (i.e., AlexNet, ResNet-34. And MobileNet-V1) with different computational complexity as DNN-based applications, which perform well in the field of image classification.

These DNN architectures are trained on the dataset ImageNet and are implemented using python3.8 and Pytorch1.8.0.

4.1.4. Benchmark of latency prediction models

To evaluate the prediction performance of our DNN inference acceleration framework, we compare our latency prediction models based on neural networks with the following prediction approaches on different computing platforms.

- **Linear regression (LR)** [33]. This linear regression-based prediction model is a status quo approach in [6,14,34], which performs better than logistic-based prediction models.
- **Support vector machine (SVM)** [35]. This SVM-based prediction model is a status quo approach in [6], which performs other supervised learning for classification or regression tasks.
- **K-nearest neighbors (KNN)** [36]. KNN assigns heavier weights based on the distance of the nearest neighbors, which is used in [37,38] to predict the execution delay of the DNN layer.
- **Random forestry (RF)** [39]. is a multi-decision trees based classification method, which is used in [37,38] to predict the execution delay of the DNN layer.

We use the mean absolute percentage error (MAPE) [40] as a metric to evaluate the prediction performance of latency prediction models.

$$MAPE = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{\hat{y}_i - y_i}{y_i} \right| \quad (6)$$

MAPE is used to measure the prediction error. In the experiments, the smaller the value is, the better the prediction performance is.

Table 5
Network specifications.

Network	Communication mode	Bandwidth (Mbps) Uplink/Downlink			
		Min	Max	Avg	Jitter
4G	Device-to-Network edge	12.06/29.49	15.07/32.66	13.77/30.88	3.01/3.17
5G		32.13/219.45	35.47/307.28	33.42/269.89	3.34/87.84
Wi-Fi		34.98/86.53	44.87/207.83	36.74/102.62	9.89/121.3
High-Speed optical fiber lines	Network Edge-to-Cloud	81.36/420.16	84.65/456.46	83.11/437.11	3.29/36.3

4.1.5. Benchmark of computation partitioning algorithm

To evaluate the performance of our computation partitioning algorithm, we compare our approach with the following partitioning strategy in different wireless networks.

- **End-only:** This is a local strategy, where the entire DNN computations are executed on an end device.
- **Edge-only:** This is an edge strategy, where the entire DNN computations are executed on the edge server.
- **Cloud-only:** This is a cloud strategy, where the entire DNN computations are executed on the cloud server.
- **End-edge:** This is a collaborative partitioning strategy, which is realization of Edgent [6]. Specifically, the strategy outputs the optimal one partitioning point, where the first block is executed on the end devices and the rest block is offloaded to the edge server for executions.
- **End-cloud:** This is an end-cloud computing paradigm, which is the realization of Neurosurgeon [20]. This strategy divides the DNN computations into two blocks, where the first block is assigned to be executed on the end device and the remaining block is offloaded to the cloud server for execution.
- **End-edge-cloud:** This is an end-edge-cloud collaborative partitioning strategy, which is a realization of [14,28]. The strategy leverages Deep Q-Network (DQN) to learn the optimal partitioning decision (i.e., the computing platform to which each DNN layer will be assigned).

4.2. Prediction performance analysis

In the section, we train 12 fully-connected neural networks based on four-layer types (i.e., CONV, FC, ACT, and POOL) and three computing platforms (i.e., end devices, edge servers, and the cloud server) for pre-layer latency prediction. Each layer has 32 neurons. During the training process, we set the epoch to 300, the batch size to 64, and the learning rate to 0.01.

4.2.1. Convergence performance

In the experiments, we investigate the convergence performance of latency prediction models under various layer types and computing platforms. Fig. 3 shows the convergence speed of four latency prediction models under three computing platforms. These prediction models converge at different numbers of epochs against the cloud computing platform, the edge computing platform, and the end computing platform, respectively. Specifically, the CONV latency prediction models, the FC latency prediction models, the POOL latency prediction models, and the ACT latency prediction models converge after about 96 epochs, 42 epochs, 37 epochs, and 33 epochs on average, respectively. We can see that the ACT latency prediction models converge fastest while the CONV latency prediction models have the slowest convergence speed. This is because the activation layer has fewer configuration parameters than the convolution layer. In addition, the test accuracy of these converged latency prediction models can achieve 96%, 98%, 97%, and 97% on average, respectively.

4.2.2. Prediction performance comparison

In the experiments, we compare our latency prediction approach with the other four approaches on different computing platforms and different DNNs in terms of prediction error. Our approach consistently achieves better performance than four baseline approaches (i.e., SVM, LR, KNN, and RF) under three computing platforms as shown in Figs. 4, 5, and 6.

Fig. 4 shows our approach has relatively 12.11% lower MAPE (i.e., 10.19%, 15.36%, and 10.76% for AlexNet, ResNet-34, and MobileNet-V1, respectively), 13.39% lower MAPE, 17.09% lower MAPE, and, 48.53% lower MAPE on average than the four baseline methods, including SVM, LR, KNN, and RF.

Fig. 5 presents the prediction accuracy achieved by our approach and the other four approaches on the edge computing platform. Fig. 5a, b, and c present results for a given DNN type, respectively. More specifically, our approach achieves on average 68.24% lower reduction, 71.16% lower reduction, and 79.10% lower reduction in MAPE than AlexNet, ResNet-34, and MobileNet-V1.

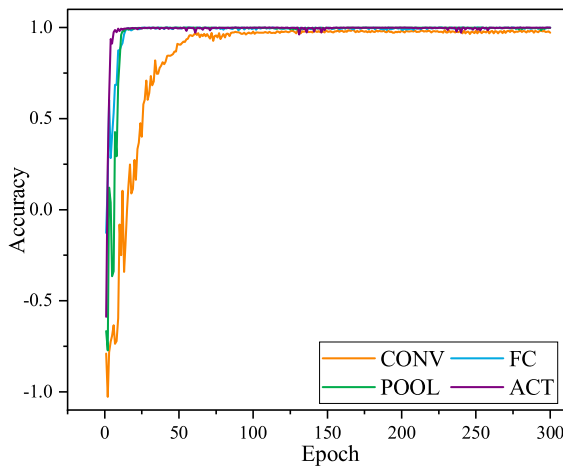
Fig. 6 shows our approach's latency improvement over the four baselines on the end computing platform. Fig. 6a shows our approach improves the prediction accuracy by about 45.84% on average for AlexNet compared with the four baseline approaches (i.e., 58.12%, 41.65%, 40.42%, and 43.16% for SVM, LR, KNN, and RF, respectively). Similar to the improvement for AlexNet, Fig. 6b shows the MAPE errors of our approach are reduced by approximately 74.10% on average (i.e., 81.27%, 75.33%, 79.65%, and 60.15% for SVM, LR, KNN, and RF, respectively) for ResNet-34. As illustrated in Fig. 6c, our approach reduces the MAPE to 0.06 on average for MobileNet-V1, which is better than the best approach RF.

These results indicate that the baseline methods do not learn well non-linear features related to inference latency. On the other hand, our approach uses neural networks with the generalization ability to efficiently capture complex non-linear relationships from the train data. Therefore, our latency prediction models have a good performance.

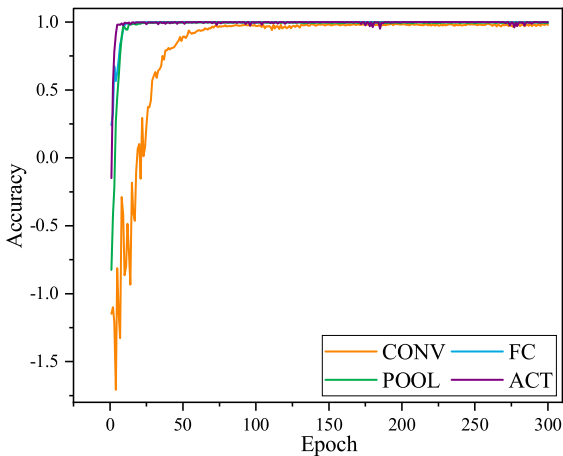
4.3. Inference latency performance analysis

4.3.1. Comparing the performance of one-point partitioning and two-point partitioning

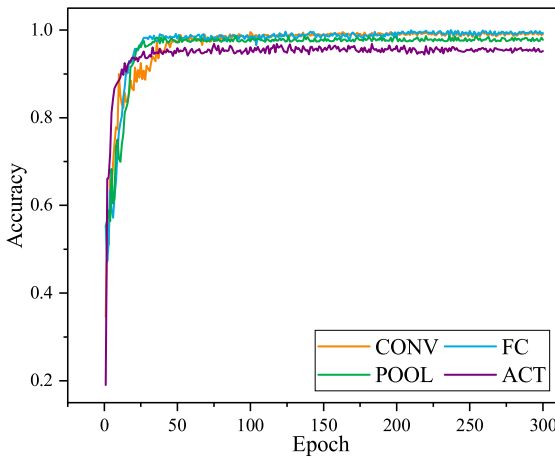
In the experiments, we show the effectiveness of our approach by comparing two-point partitioning with one-point partitioning, and the results are shown in Fig. 7. It is observed that our approach always outperforms one-point partitioning methods (i.e., the end-edge and the end-cloud). Specifically, our approach reduces the latency by about 31 ms (i.e., 3 ms, 58 ms, and 32 ms for AlexNet, ResNet-34, and MobileNet-V1, respectively), and 16.5 ms (i.e., 9 ms, 26 ms, and 14 ms for AlexNet, ResNet-34, and MobileNet-V1, respectively) on average against the end-cloud and the end-edge. This is because these two approaches fail to make a good balance between execution latency and communication latency. Therefore, we can see that our approach really works. Although our approach brings extra communication overhead between the edge server and the cloud, two-point partitioning can better accelerate DNN inference than one-point partitioning in an end-edge-cloud collaborative computing environment by greatly reducing execution latency.



(a) The convergenc performance of four latency prediction models on the cloud computing platform



(b) The convergenc performance of four latency prediction models on the edge computing platform



(c) The convergenc performance of four latency prediction models on the end computing platform

Fig. 3. The convergence performance under different layer types and different computing platforms.

4.3.2. Network variation

Fig. 8 shows the results of our approach and the other six baselines under three wireless networks (i.e., 4G, 5G, and Wi-Fi). It can be observed that our approach can achieve better performance than the baseline methods in terms of end-to-end latency. More specifically, as the average uplink and downlink

bandwidths increase from 4G (i.e., 13.77/30.88 Mbps) to Wi-Fi (i.e., 36.74/102.62 Mbps), various approaches can consistently reduce the latency except for the end-only approach. This is because its inference latency only depends on local execution latency. In addition, to enable the figure to show details, Fig. 8 only shows the result of our approach and the other five baseline approaches (i.e., cloud-only, edge-only, end-cloud, and end-edge, end-edge-cloud) due to the extensive processing latency of end-only. Taking MobileNet-V1 as an example, the processing latencies of end-only is 4096.09 ms, while our approach on 4G, 5G, and Wi-Fi are 91.20 ms, 42.16 ms, and 37.08 ms, respectively.

Then, we compare our approach with the three baseline methods without partitioning points (i.e., the end-only, the cloud-only, and the edge-only). On average our approach has relatively about 26.46% lower latency, 40.56% lower latency, and 41.44% lower latency than three DNNs (i.e., AlexNet, ResNet-34, and MobileNet-V1) on 4G, 5G, and Wi-Fi. The reason is that these baseline approaches limit the entire DNN computations to be executed either on the device or be offloaded to the edge server or the cloud server for execution. Obviously, these approaches fail to take collaborative computing into consideration. Finally, we compare our approach against the remaining two baseline methods with one partitioning point (i.e., the end-edge and the end-cloud). Although these two approaches can reduce the end-to-end latency through collaborative computing between devices and edge servers or between devices and the cloud server, they perform worse than our approach. This is because these two approaches fail to take full use of the end-edge-cloud computing platforms. On the contrary, our approach adopts two partitioning points to make a good balance between communication latency and execution time, where the DNN block computation can be assigned to the device, or the edge server, or the cloud server for execution. Taking ResNet-34 as an example, compared with the end-edge and the end-cloud, our approach reduces the latency by about 9.25 ms, 10.53 ms, and 11.52 ms on average for 4G, 5G, and Wi-Fi, respectively.

Finally, we compare our approach with the remaining end-edge-cloud approach based on DQN, and the results show that this approach performs worse than our approach. Taking MobileNet-V1 as an example, compared to the end-edge-cloud, our approach reduces the latency by about 3.12 ms, 2.53 ms, and 4.43 ms on average for 4G, 5G, and Wi-Fi, respectively. The main reason is that compared with execution latency, the end-edge-cloud approach based on DQN brings more communication overhead.

In addition, when wireless networks have enough fast uplink and downlink bandwidths, the end-edge approach or the end-cloud approach eventually evolves into the edge-only approach or the cloud-only approach, respectively. That is, the end-to-end latency of the end-edge approach will be equivalent to that of the edge-only approach in case the network speed is fast enough. This is because the communication latency between devices and edge servers or between devices and the cloud server is less than the local execution time in the fast network condition.

To further verify the adaptability of our approach, we conduct experiments with different communication rates on Wi-Fi executing AlexNet, and the results are shown in Fig. 9. As the communication rate increases from 3 Mbps to 30 Mbps, our approach always outperforms the other five baseline approaches in terms of end-to-end latency.

When the communication rate is lower than 3 Mbps, all collaborative computing approaches (i.e., the end-edge approach, the end-cloud approach, the end-edge-cloud, and our approach) evolve into the end-only approach. That is, these collaboration approaches choose to execute the entire DNN computations locally without offloading computation. This is because the communication latency caused by the low network bandwidth is

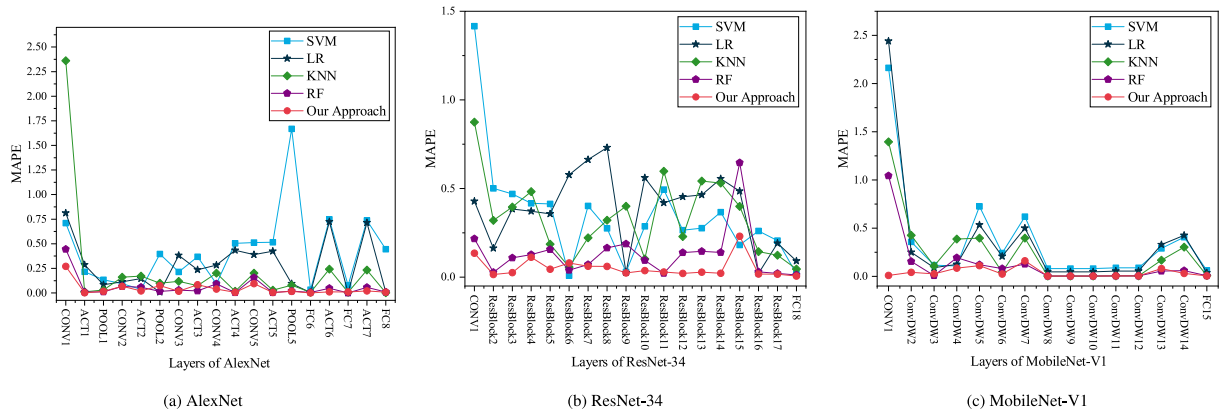


Fig. 4. Each layer's prediction errors on the cloud computing platform.

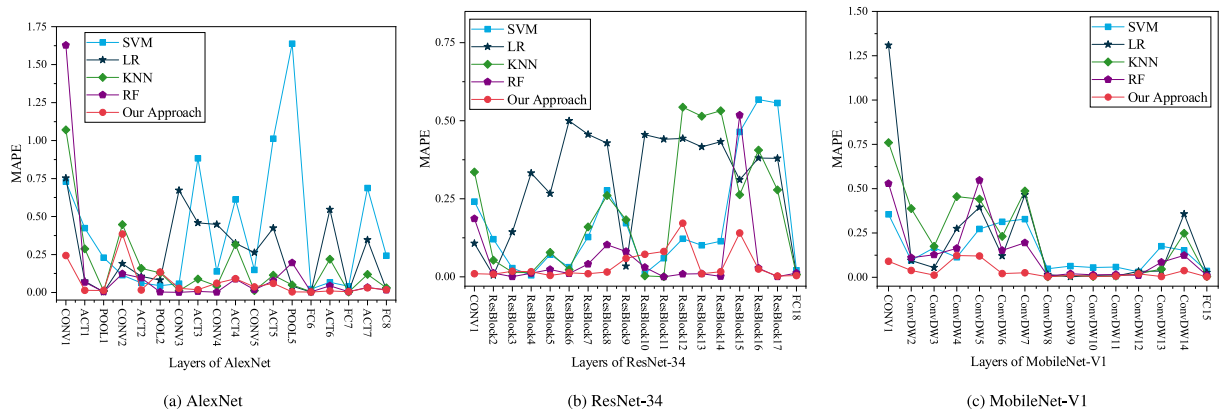


Fig. 5. Each layer's prediction errors on the edge computing platform.

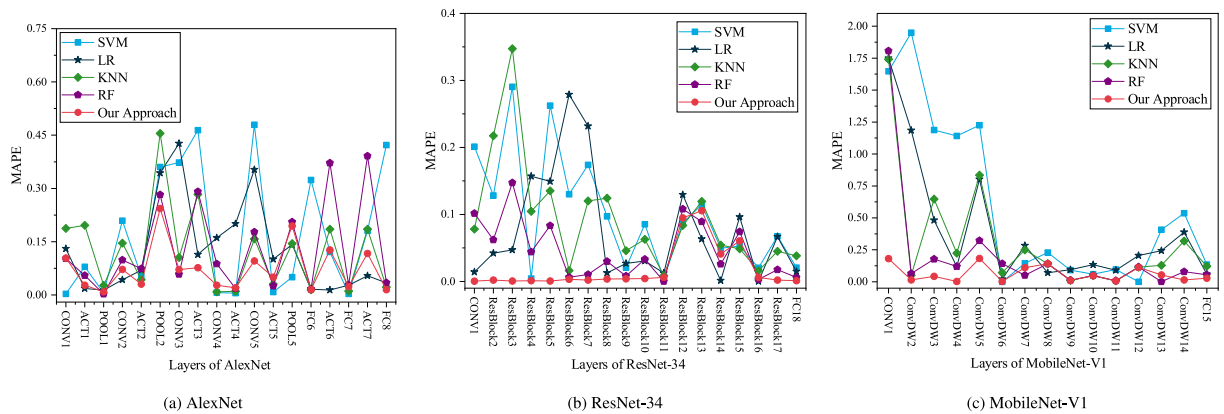


Fig. 6. Each layer's prediction errors on the end computing platform.

much greater than the local execution inference. Therefore, all collaboration approaches evolve into the end-only approach.

When the communication rate is lower than 8Mbps, the inference latency of the cloud-only approach and the edge-only approach is much higher than the end-only approach. This indicates that offloading the entire DNN computations to the edge server or the cloud server for accelerating inference in the low network condition is not feasible.

When the communication rate ranges from 3 Mbps to 13 Mbps, our approach chooses to divide the DNN computations into

two blocks using one partitioning point. The data-intensive block is executed on the end devices while the rest block is assigned to the edge server for execution. In this situation, our approach evolves into the end-edge approach. That is, our approach and the end-edge approach achieve a lower end-to-end inference than the other four approaches. This is because as the network bandwidth increases, the end-edge communication latency reduces. This indicates that all collaborative computing approaches can accelerate DNN inference through offloading computation in case the network bandwidth is not too low.

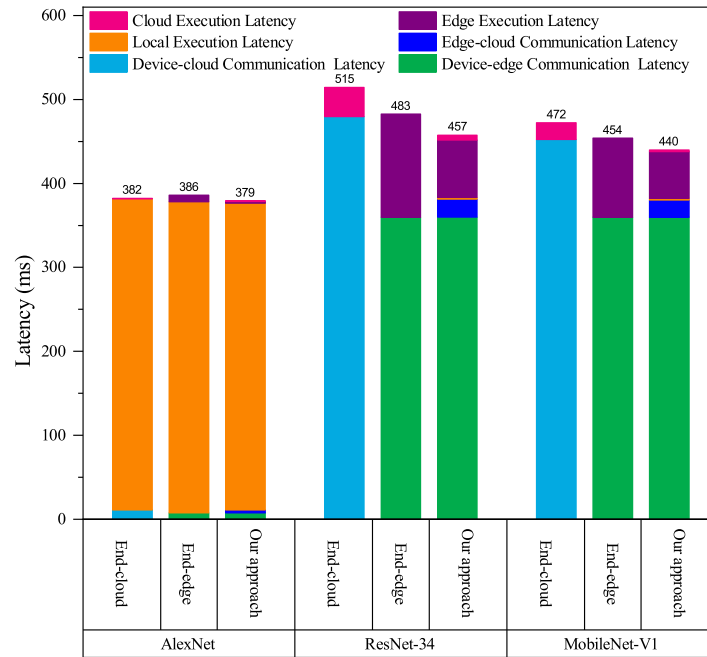


Fig. 7. Comparison of the performance of one-point partitioning and two-point partitioning.

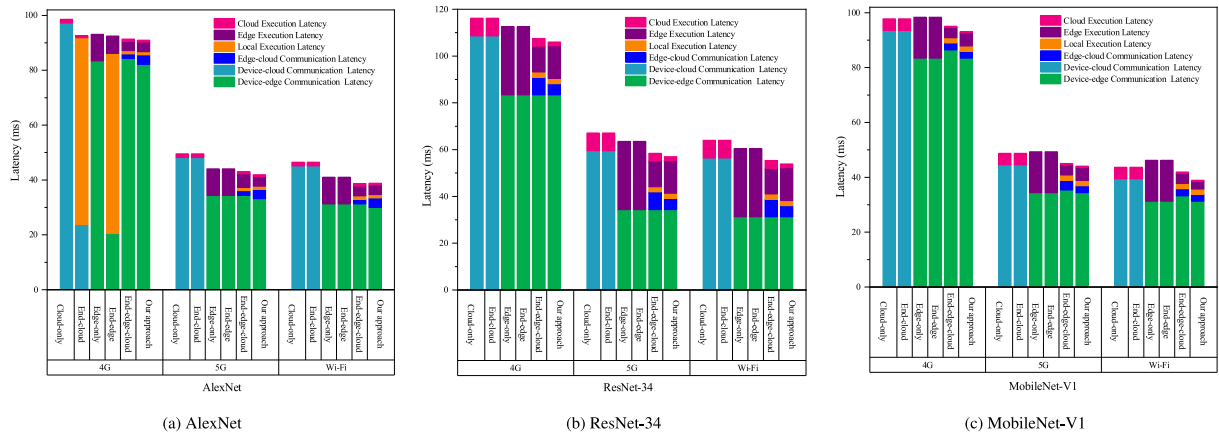


Fig. 8. Comparison of our approach with other baselines under three wireless networks.

When the communication rate is higher than 13 Mbps, our approach chooses to divide the DNN computation into three blocks using two partitioning points. Specifically, the data-intensive block is executed on the end devices, the hybrid block is completed on the edge server, and the computation-intensive block is offloaded to the cloud server. Through offloading computation, our approach makes a good balance between the network bandwidth and the end-edge-cloud computing platforms. Therefore, our approach achieves the minimum end-to-end inference.

4.3.3. Different sizes of input images

In the experiments, we evaluate how our approach makes an adaptive partitioning strategy with different input image sizes on Wi-Fi, and the results are shown in Fig. 10. As the input image size increase from 224×224 to 760×760 and the average uplink and downlink bandwidth of Wi-Fi are set to 36.74Mbps and 102.6Mbps, respectively, the end-to-end latency of various approaches increases. This is because the larger input image not only increases the execution latency but also increases the

communication latency. However, our approach outperforms the other six baseline approaches under different input image sizes in terms of end-to-end inference. In addition, to enable the figure to show details, Fig. 10 only shows the result of our approach and the other five baseline approaches (i.e., cloud-only, edge-only, end-cloud, and end-edge, end-edge-cloud) due to the extensive processing latency of end-only. Taking ResNet-34 as an example, the processing latencies of end-only on 224×224 , 460×460 , 640×640 , and 760×760 are 937.26 ms, 3639.69 ms, 6323.59 ms, and 9689.01 ms, respectively, while our approach are 51.88 ms, 185.22 ms, 337.69 ms, and 457.48 ms.

In Fig. 10a, we compare our approach with the other five baselines with different input image sizes on AlexNet in terms of end-to-end latency. On average our approach has relatively about 22.58% lower latency, 3.07% lower latency, 7.65% lower latency, 2.18% lower latency, and 0.46% lower latency than the five baseline methods (i.e., the cloud-only, the edge-only, the end-cloud, the end-edge, the end-edge-cloud). Specifically, when the

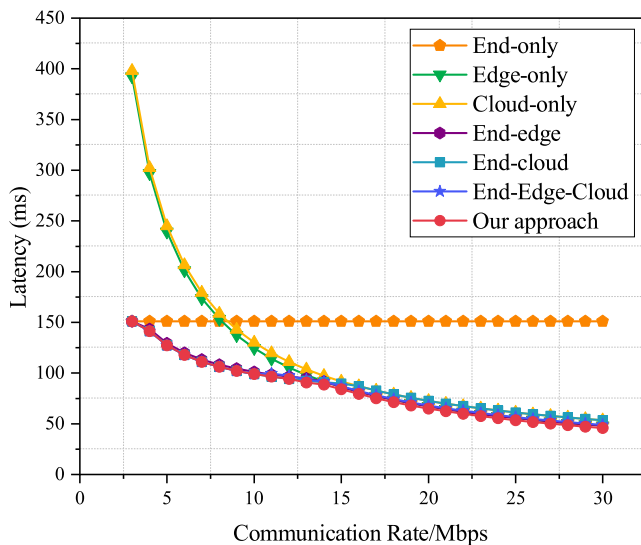


Fig. 9. Comparison of our approach with other baselines under different communication rates.

input image sizes are set to 224×224 , the latency of the end-edge approach or the end-cloud approach is equivalent to that of the edge-only approach or the cloud-only approach, respectively. This is because the transmission delay caused by the raw data is smaller than the local inference latency. Then, when the input image sizes are increased to 460×460 and 640×640 , the performance of our approach is improved by 23.17%, 1.19%, 6.75%, 1.19%, and 0.54% on average compared with the cloud-only, the edge-only, the end-cloud, the end-edge, and the end-edge-cloud, respectively. This is because our approach makes a good balance between the communication latency and the execution latency through end-edge-cloud collaborative computing. In addition, it should be noted that the end-cloud approach is better than the cloud-only approach. This indicates that as the input image size increases, the end-cloud approach begins to consider collaborative computing between devices and the cloud server by executing some computations locally. Finally, when the input image size is set to 760×760 , our approach reduces the latency by about 144.36 ms, 21.08 ms, 3.81 ms, 6.58 ms, and 2.23 ms on average against the cloud-only, the edge-only, the end-cloud, the end-edge, and the end-edge-cloud, respectively. The reason is that our approach can fully utilize the computing resources of end devices, edge servers, and the cloud server through two-point partitioning mechanism. It also should be noted that the end-edge approach begins to consider collaborative computing between devices and edge servers through one-point partitioning mechanism. This indicates that the end-edge approach chooses to execute some layers locally as the input image size increases.

Fig. 10b presents the end-to-end latency achieved by our approach and the other five baselines with different input image sizes on ResNet-34. It is observed that our approach always outperforms the baseline methods. Specifically, our approach achieves on average 8.56% lower reduction (i.e., 7.39%, 8.47%, 7.28%, and 11.09% for 224×224 , 460×460 , 640×640 , and 760×760 , respectively), 7.97% lower reduction, 8.56% lower reduction, 7.97% lower reduction, and 3.35% lower reduction in end-to-end latency than the cloud-only, the edge-only, the end-cloud, the end-edge, and the end-edge-cloud. This indicates that our approach can make a good balance between the communication latency and the execution latency by dividing ResNet-34 computations into end devices, edge servers, and the cloud server. In addition, it should be noted that these collaborative computing approaches (i.e., the end-edge and the end-cloud) do

not consider the computing resources of end devices when executing ResNet-34. The reason is that it is difficult to execute ResNet-34 layers on resource-limited end devices with low latency as ResNet-34 with complex network structure needs more computing cost than AlexNet.

Similar to Fig. 10b, c shows our approach improves the performance by 6.07% on average for MobileNet-V1 compared with the five baseline approaches (i.e., 6.97%, 5.81%, 6.97%, 5.81%, and 4.78% for the cloud-only, the edge-only, the end-cloud, the end-edge, and the end-edge-cloud, respectively) with different input image sizes.

5. Conclusion

In this paper, we have presented an adaptive DNN inference acceleration framework to reduce DNN inference latency in the end-edge-cloud computing environment. In this framework, we first built latency prediction models based on neural networks for DNN layer execution, which provide a basis for DNN computation partitioning. Then, we designed a computation partitioning algorithm based on latency prediction results and network conditions for accelerating DNN inference, which divides DNN computations into end devices, edge servers, and the cloud server. Experimental results show that our approach can effectively reduce the end-to-end inference of DNN-based intelligent applications and outperforms other baseline approaches.

In future work, we will take into account how to coordinate the balance between latency and energy consumption in our framework for further accelerating DNN inference. In addition, a fine-grained DNN partitioning based on layer is needed to take into account.

CRedit authorship contribution statement

Guozhi Liu: Writing – original draft, Conceptualization, Methodology, Software, Visualization, Data curation. **Fei Dai:** Conceptualization, Methodology, Writing – review & editing, Supervision, Funding acquisition, Validation. **Xiaolong Xu:** Writing – review & editing, Resources, Formal analysis. **Xiaodong Fu:** Supervision, Validation. **Wanchun Dou:** Funding acquisition, Validation. **Neeraj Kumar:** Editing, Resources. **Muhammad Bilal:** Writing – review & editing, Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgments

This work has been supported by the Project of National Natural Science Foundation of China under Grant No. 62262063, the Project of Key Science Foundation of Yunnan Province, China under Grant No. 202101AS070007, Dou Wanchun Expert Workstation of Yunnan Province, China No. 202205AF150013, Science and Technology Youth lift talents of Yunnan Province, China, and the Project of Scientific Research Fund Project of Yunnan Education Department, China under Grant No. 2022Y561.

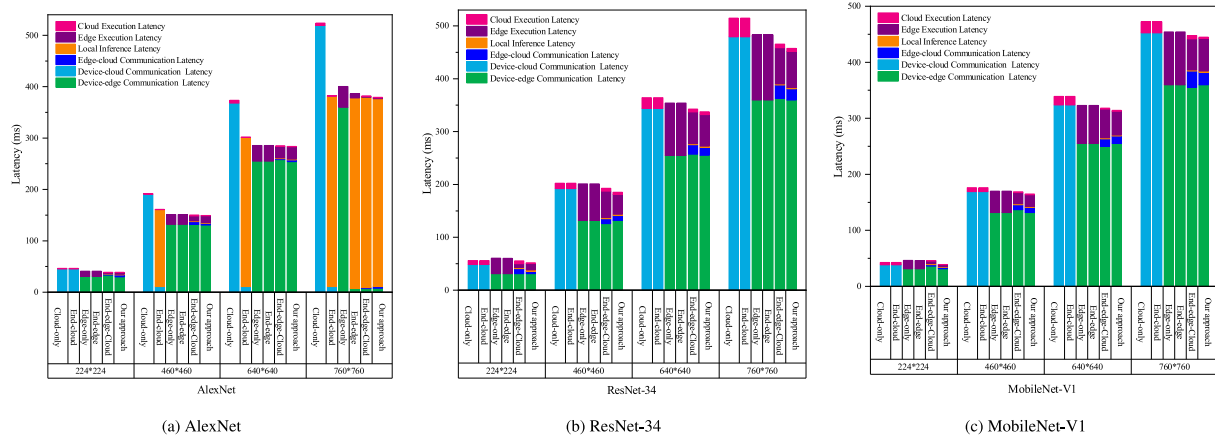


Fig. 10. Comparison of our approach with other baseline approaches under different input image sizes.

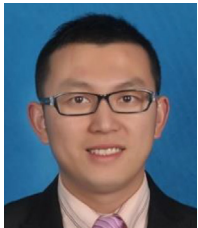
References

- [1] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, *Adv. Neural Inf. Process. Syst.* 25 (2012).
- [2] S. Zhang, L. Yao, A. Sun, Y. Tay, Deep learning based recommender system: A survey and new perspectives, *ACM Comput. Surv.* 52 (1) (2019) 1–38.
- [3] A.M. Anything, Dynamic memory networks for natural language processing, 2015, Kumar et al. ArXiv Pre-Print 97.
- [4] A. Banitalebi-Dehkordi, N. Vedula, J. Pei, F. Xia, L. Wang, Y. Zhang, Auto-split: a general framework of collaborative edge-cloud ai, in: *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 2021, pp. 2543–2553.
- [5] N.D. Lane, S. Bhattacharya, A. Mathur, C. Forlivesi, F. Kawsar, DXTK: Enabling resource-efficient deep learning on mobile and embedded devices with the deepx toolkit, in: *MobiCASE*, 2016, pp. 98–107.
- [6] E. Li, L. Zeng, Z. Zhou, X. Chen, Edge AI: On-demand accelerating deep neural network inference via edge computing, *IEEE Trans. Wireless Commun.* 19 (1) (2019) 447–457.
- [7] M. Xue, H. Wu, R. Li, M. Xu, P. Jiao, EosDNN: An efficient offloading scheme for DNN inference acceleration in local-edge-cloud collaborative environments, *IEEE Trans. Green Commun. Netw.* (2021).
- [8] Y. Huang, X. Qiao, P. Ren, L. Liu, C. Pu, S. Dustdar, J. Chen, A lightweight collaborative deep neural network for the mobile web in edge cloud, *IEEE Trans. Mob. Comput.* (2020).
- [9] W. Xiao, Y. Miao, G. Fortino, D. Wu, M. Chen, K. Hwang, Collaborative cloud-edge service cognition framework for DNN configuration toward smart IoT, *IEEE Trans. Ind. Inform.* (2021).
- [10] A.E. Eshratifar, M.S. Abrishami, M. Pedram, JointDNN: an efficient training and inference engine for intelligent mobile cloud computing services, *IEEE Trans. Mob. Comput.* 20 (2) (2019) 565–576.
- [11] A.E. Eshratifar, A. Esmaili, M. Pedram, Bottlenet: A deep learning architecture for intelligent mobile cloud computing services, in: *2019 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*, IEEE, 2019, pp. 1–6.
- [12] E. Li, Z. Zhou, X. Chen, Edge intelligence: On-demand deep learning model co-inference with device-edge synergy, in: *Proceedings of the 2018 Workshop on Mobile Edge Communications*, 2018, pp. 31–36.
- [13] M. Gao, R. Shen, L. Shi, W. Qi, J. Li, Y. Li, Task partitioning and offloading in DNN-task enabled mobile edge computing networks, *IEEE Trans. Mob. Comput.* (2021).
- [14] P. Ren, X. Qiao, Y. Huang, L. Liu, C. Pu, S. Dustdar, Fine-grained elastic partitioning for distributed DNN towards mobile web AR services in the 5G era, *IEEE Trans. Serv. Comput.* (2021).
- [15] Z. Gao, D. Miao, L. Zhao, Z. Mo, G. Qi, L. Yan, Triple-partition network: Collaborative neural network based on the ‘end device-edge-cloud’, in: *2021 IEEE Wireless Communications and Networking Conference (WCNC)*, IEEE, 2021, pp. 1–7.
- [16] C. Jiang, J. Wan, A thing-edge-cloud collaborative computing decision-making method for personalized customization production, *IEEE Access* 9 (2021) 10962–10973.
- [17] S. Chinchali, A. Sharma, J. Harrison, A. Elhafsi, D. Kang, E. Pergament, E. Cidon, S. Katti, M. Pavone, Network offloading policies for cloud robotics: a learning-based approach, *Auton. Robots* 45 (7) (2021) 997–1012.
- [18] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, J. Zhang, Edge intelligence: Paving the last mile of artificial intelligence with edge computing, *Proc. IEEE* 107 (8) (2019) 1738–1762.
- [19] L. Zhou, M.H. Samavatian, A. Bacha, S. Majumdar, R. Teodorescu, Adaptive parallel execution of deep neural networks on heterogeneous edge devices, in: *Proceedings of the 4th ACM/IEEE Symposium on Edge Computing*, 2019, pp. 195–208.
- [20] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, L. Tang, Neurosurgeon: Collaborative intelligence between the cloud and mobile edge, *ACM SIGARCH Comput. Archit. News* 45 (1) (2017) 615–629.
- [21] E. Baccarelli, S. Scardapane, M. Scarpiniti, A. Momenzadeh, A. Uncini, Optimizing and scalable implementation of conditional deep neural networks with early exits for fog-supported IoT applications, *Inform. Sci.* 521 (2020) 107–143.
- [22] Y. Duan, J. Wu, Joint optimization of DNN partition and scheduling for mobile cloud computing, in: *50th International Conference on Parallel Processing*, 2021, pp. 1–10.
- [23] E. Baccarelli, M. Scarpiniti, A. Momenzadeh, S.S. Ahrabi, Learning-in-the-fog (LiFo): Deep learning meets fog computing for the minimum-energy distributed early-exit of inference in delay-critical IoT realms, *IEEE Access* 9 (2021) 25716–25757.
- [24] A.E. Eshratifar, M. Pedram, Energy and performance efficient computation offloading for deep neural networks in a mobile cloud computing environment, in: *Proceedings of the 2018 on Great Lakes Symposium on VLSI*, 2018, pp. 111–116.
- [25] E. Baccarelli, M. Scarpiniti, A. Momenzadeh, Ecomobifog—design and dynamic optimization of a 5g mobile-fog-cloud multi-tier ecosystem for the real-time distributed execution of stream applications, *IEEE Access* 7 (2019) 55565–55608.
- [26] L. Zeng, X. Chen, Z. Zhou, L. Yang, J. Zhang, Coedge: Cooperative dnn inference with adaptive workload partitioning over heterogeneous edge devices, *IEEE/ACM Trans. Netw.* 29 (2) (2020) 595–608.
- [27] H.-J. Jeong, H.-J. Lee, C.H. Shin, S.-M. Moon, IONN: Incremental offloading of neural network computations from mobile devices to edge servers, in: *Proceedings of the ACM Symposium on Cloud Computing*, 2018, pp. 401–411.
- [28] W. Zhang, D. Yang, H. Peng, W. Wu, W. Quan, H. Zhang, X. Shen, Deep reinforcement learning based resource management for DNN inference in industrial IoT, *IEEE Trans. Veh. Technol.* 70 (8) (2021) 7605–7618.
- [29] B. Lin, Y. Huang, J. Zhang, J. Hu, X. Chen, J. Li, Cost-driven off-loading for DNN-based applications over cloud, edge, and end devices, *IEEE Trans. Ind. Inform.* 16 (8) (2019) 5456–5466.
- [30] P. Ren, X. Qiao, Y. Huang, L. Liu, S. Dustdar, J. Chen, Edge-assisted distributed DNN collaborative computing approach for mobile web augmented reality in 5G networks, *IEEE Netw.* 34 (2) (2020) 254–261.
- [31] S. Yu, X. Chen, L. Yang, D. Wu, M. Bennis, J. Zhang, Intelligent edge: Leveraging deep imitation learning for mobile edge computation offloading, *IEEE Wirel. Commun.* 27 (1) (2020) 92–99, <http://dx.doi.org/10.1109/MWC.001.1900232>.
- [32] speedtest, <https://www.speedtest.cn/>.
- [33] G. Seber, A. Lee, Linear Regression Analysis, in: *Wiley Series in Probability and Statistics*, Wiley, 2012, URL <https://books.google.com.hk/books?id=X2Y60kXl8ysc>.
- [34] H. Li, C. Hu, J. Jiang, Z. Wang, Y. Wen, W. Zhu, JALAD: Joint accuracy-and-latency-aware deep structure decoupling for edge-cloud execution, 2018, pp. 671–678, <http://dx.doi.org/10.1109/PADSW.2018.8645013>.
- [35] S. Suthaharan, Support Vector Machine, Springer US, Boston, MA, 2016, pp. 207–235, http://dx.doi.org/10.1007/978-1-4899-7641-3_9.
- [36] S.A. Dudani, The distance-weighted k-nearest-neighbor rule, *IEEE Trans. Syst. Man Cybern.* SMC-6 (4) (1976) 325–327, <http://dx.doi.org/10.1109/TSMC.1976.5408784>.

- [37] S. Sun, X. Li, M. Liu, B. Yang, X. Guo, DNN inference acceleration via heterogeneous IoT devices collaboration, *Jisuanji Yanjiu Yu Fazhan/Comput. Res. Dev.* 57 (4) (2020) 709–722.
- [38] S. Yao, Y. Zhao, H. Shao, S. Liu, D. Liu, L. Su, T. Abdelzaher, Fastdeepiot: Towards understanding and optimizing neural network execution time on mobile and embedded devices, in: *Proceedings of the 16th ACM Conference on Embedded Networked Sensor Systems*, 2018, pp. 278–291.
- [39] A. Cutler, D.R. Cutler, J.R. Stevens, in: C. Zhang, Y. Ma (Eds.), *Random Forests*, Springer US, Boston, MA, 2012, pp. 157–175, http://dx.doi.org/10.1007/978-1-4419-9326-7_5.
- [40] S. Yao, Y. Zhao, H. Shao, S. Liu, D. Liu, L. Su, T. Abdelzaher, FastDeepIoT: Towards understanding and optimizing neural network execution time on mobile and embedded devices, *SenSys '18*, Association for Computing Machinery, New York, NY, USA, 2018, pp. 278–291, <http://dx.doi.org/10.1145/3274783.3274840>.

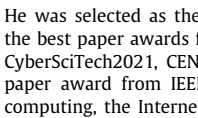


Guozhi Liu received the Bachelor's degree from Southwest Forestry University, Kunming, China. He is currently working toward the Master's degree at the School of Big Data and Intelligence Engineering, Southwest Forestry University, Kunming, China. His research interests include deep learning, deep reinforcement learning, cloud computing and mobile edge computing.



IEEE International Conference on Web Services, etc. He is the recipient of the Best Paper Award of IEEE International Conference on Cloudcomp 2019.

Fei Dai received the Ph.D. degree from Yunnan University, Kunming, China, in 2011. He is currently a Full Professor with the School of Big Data and Intelligence Engineering, Southwest Forestry University, Kunming, China. His research interests include edge computing, business process management, and software engineering. He has authored and co-authored more than 80 peer review papers in international journals and conferences including IEEE Transactions on Industrial Informatics, IEEE Internet of Things Journal, World Wide Web Journal, Software: Practice and Experience, IEEE International Conference on Web Services, etc. He is the recipient of the Best Paper Award of IEEE International Conference on Cloudcomp 2019.



He was selected as the Highly Cited Researcher of Clarivate 2021. He received the best paper awards from IEEE CBD 2016, IEEE CPSCoM 2020, SPDE 2020, IEEE CyberSciTech2021, CENET 2021, and EAI Cloudcomp 2021, and the distinguish paper award from IEEE Smart City 2021. His research interests include edge computing, the Internet of Things (IoT), cloud computing, and big data.

Xiaolong Xu received the Ph.D. degree in computer science and technology from Nanjing University, China, in 2016. He was a Research Scholar with Michigan State University, USA, from April 2017 to May 2018. He is currently a Full Professor with the School of Computer and Software, Nanjing University of Information Science and Technology. He has published more than 100 peer-review articles in international journals and conferences, including the IEEE TSC, IEEE TFS, IEEE TKDE, IEEE TPDs, IEEE T-ITS, IEEE TII, ACM TOSN, ACM TOMM, ACM TIST, ACM SIGIR, IEEE ICWS, ICSOC, etc.



Xiaodong Fu received his MS degree in Computer Science and his Ph.D. degree in Management Science from Kunming University of Science and Technology in 2000 and 2008 respectively. He is currently a professor at the Faculty of Information Engineering and Automation, Kunming University of Science and Technology, China. His current research interests include services computing and intelligent decision making. He is now a senior member of China Computer Federation and a member of China Computer Federation Technical Committee on Service Computing.



papers in international journals and international conferences. His research interests include workflow, cloud computing, and service computing.

Wanchun Dou (Member, IEEE) received the Ph.D. degree in mechanical and electronic engineering from the Nanjing University of Science and Technology, Nanjing, China, in 2001. From April 2005 to June 2005 and from November 2008 to February 2009, he visited the Department of Computer Science and Engineering, Hong Kong University of Science and Technology, Hong Kong, as a Visiting Scholar. He is currently a Full Professor with the State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing. He has authored or coauthored more than 100 research papers in international journals and international conferences. His research interests include workflow, cloud computing, and service computing.



Neeraj Kumar received his Ph.D. in CSE from SMVD University, Katra (J & K), India, and was a postdoctoral research fellow in Coventry University, Coventry, UK. He is working as an Associate Professor in the Department of Computer Science and Engineering, Thapar Institute of Engineering & Technology, Patiala (Pb.), India since 2014. Dr. Neeraj is an internationally renowned researcher in the areas of VANET & CPS Smart Grid & IoT Mobile Cloud computing & Big Data and Cryptography. He has published more than 150 technical research papers in leading journals and conferences from IEEE, Elsevier, Springer, John Wiley, and Taylor and Francis. His paper has been published in some of the high impact factors journals such as IEEE Transactions on Industrial Informatics, IEEE Transactions on Industrial Electronics, IEEE Transactions on Information Forensics and Security, IEEE Transactions on Dependable and Secure Computing, IEEE Transactions on Power Systems, IEEE Transactions on Vehicular Technology, IEEE Transactions on Smart Grid, IEEE Journal of Biomedical and Health Informatics, IEEE Access, IEEE Transactions on Consumer Electronics, IEEE Systems Journal, IEEE IoT Journal, IEEE Wireless Communication Magazine, IEEE Vehicular Technology Magazine, IEEE Communication Magazine, IEEE Networks Magazine etc. Apart from the journals conferences, he has also published papers in some of the core conferences of his area of specialization such as IEEE Globecom, IEEE ICC, IEEE Greencom, IEEE CSCWD. He has guided many research scholars leading to Ph.D. and M.E./M.Tech. His research is supported by funding from TCS, CSIT, UGC and UGC in the area of Smart grid, energy management, VANETs, and Cloud computing. He is member of the Cyber-Physical Systems and Security (CPSS) research group. He has research funding from DST, CSIR, UGC, and TCS. He has total research funding from these agencies of more than 2 Crores under different schemes from the GOI. Recently, he has also got International research projects under DST-research initiative. He has h-index of 26 (according to Google scholar, March 2017) with 2500 citations to his credit. He is editorial board members of International Journal of Communication Systems, Wiley, Security and Communication, John Wiley, and Journal of Networks and Computer Applications, Elsevier.



Muhammad Bilal (Senior Member, IEEE) received the Ph.D. degree in information and communication network engineering from the School of Electronics and Telecommunications Research Institute (ETRI), Korea University of Science and Technology, in 2017. From 2017 to 2018, he was with Korea University, where he was a Postdoctoral Research Fellow with the Smart Quantum Communication Center. In 2018, he joined Hankuk University of Foreign Studies, South Korea, where he is currently working as an Associate Professor with the Division of Computer and Electronic Systems Engineering. He is the author/coauthor of several articles published in renowned journals including IEEE IoT Journal, IEEE systems Journal, IEEE Transactions on Industrial Informatics, IEEE Transactions on Intelligent Transportation System, IEEE Transactions of Vehicular Technology, and IEEE Transactions on Network Science and Engineering, etc. His research interests include cyber security, Internet of Things, named data networking, Metaverse, Artificial Intelligence, and Edge Computing. He was a Technical Program Committee Member on many international conferences, including the IEEE VTC, the IEEE ICC, and the IEEE CCNC. He serves as an Editor for the IEEE Future Directions Ethics and Policy in Technology Newsletter and the IEEE Internet Policy Newsletter.